

Implementation Document

for

KaamSetu

- Domestic Work Marketplace

Version 1.0

Prepared by

Group #: 15

Group Name: Null Pointer

Name	Roll No.	IITK Email
Ashish Gautam	240210	ashishu24@iitk.ac.in
Chaniyara Yug	240299	chaniyaray24@iitk.ac.in
Harsh Vijay	240436	harshvijay24@iitk.ac.in
Harshit Saini	240442	harshits24@iitk.ac.in
Himanshu Yadav	240455	himanshuy24@iitk.ac.in
Ninad Tagade	240699	ninadt24@iitk.ac.in
Nirajkumar Binod Prasad	240701	nirajkp24@iitk.ac.in
Piyush Santosh Ghayal	240748	santosh24@iitk.ac.in
Vaidik	241130	vaidik24@iitk.ac.in
Virag Rathod	241170	virag24@iitk.ac.in

Course: CS253

Mentor TA: Utkarsh Agarwal

Date: 27/03/2026

Contents

CONTENTS	1
REVISIONS	2
1 IMPLEMENTATION DETAILS	3
2 CODEBASE.....	7
• FRONT-END	
• BACK-END	
• DATABASES	
• API CALLS	
3 COMPLETENESS.....	15
APPENDIX A - GROUP LOG	20

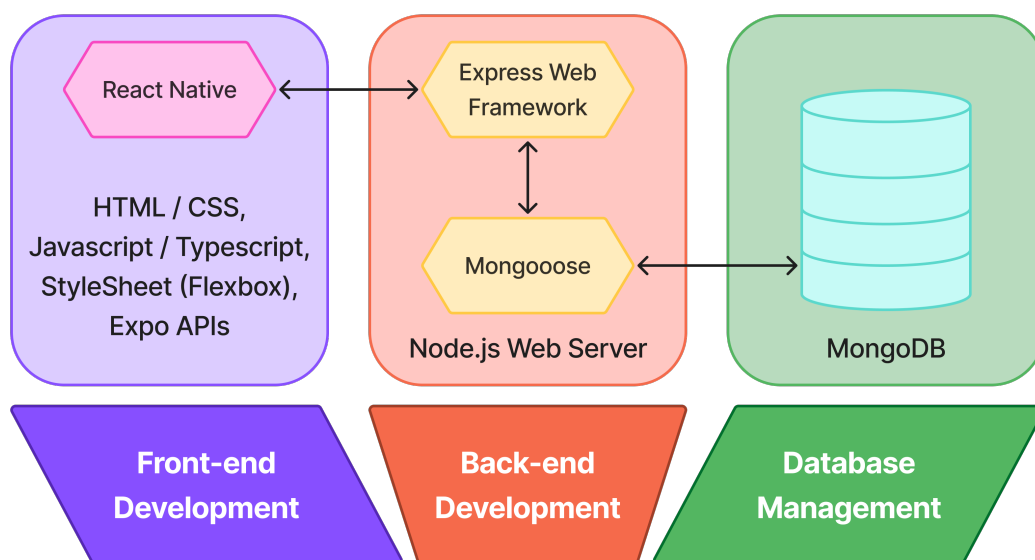
Revisions

Version	Primary Author(s)	Description of Version	Date Completed
v1.0	Ashish Gautam Chaniyara Yug Harsh Vijay Harshit Saini Himanshu Yadav Ninad Tagade Nirajkumar Binod Prasad Piyush Santosh Ghayal Vaidik Virag Rathod	The first complete draft of the KaamSetu Implementation Document. This version includes the system architecture, detailed technology justifications, codebase structure, and feature completeness report.	26/03/2026

1 Implementation Details

System Architecture and Overview

KaamSetu is implemented using a decoupled, three-tier Client-Server architecture designed to provide a seamless marketplace experience on mobile devices. The architecture separates the presentation layer from the business logic and data persistence layers, allowing for independent scaling and maintenance of the frontend and backend components.



Programming Languages

The foundational logic of the system is built upon a unified JavaScript-based ecosystem, enabling efficient full-stack development.

TypeScript

TypeScript is utilized as the primary development language for both the React Native frontend and the Node.js backend.

- **Type Safety:** By enforcing static typing, TypeScript allows the team to catch data-structure errors during development, significantly reducing runtime crashes in the mobile environment.
- **Developer Productivity:** Features like advanced IDE auto-completion and self-documenting code are essential for maintaining consistency across a large team of ten developers.

- **Alternative Comparison:** Compared to vanilla JavaScript, TypeScript provides the structural rigor necessary for a complex marketplace with multiple user roles and intricate data models.

JavaScript (ES6+)

Standard JavaScript is used within the backend runtime for scripting and asynchronous operations. Modern ES6+ syntax, including `async/await` and destructuring, ensures that the codebase remains clean and non-blocking.

Frontend Frameworks and Libraries

The frontend is designed to deliver a high-performance, native-like experience on Android devices while maintaining cross-platform compatibility.

React Native and Expo

The mobile client is built using React Native and managed through the Expo ecosystem.

- **Single Codebase:** React Native enables the development of a single logic layer that renders native UI components, ensuring feature parity across different devices without the need for separate Swift or Kotlin repositories.
- **Expo Tooling:** Expo abstracts complex native build configurations and provides **Expo Router**, a file-system-based routing library that makes application navigation highly intuitive and structured.
- **Alternative Comparison:** Unlike hybrid web-view solutions, React Native provides superior performance and smoother animations, which are critical for an engaging user experience in a job-discovery feed.

React Hooks (State Management)

Local and global application states are managed using built-in React Hooks (`useState`, `useEffect`, `useContext`).

- **Performance:** Using hooks provides a lightweight alternative to Redux, reducing memory overhead and complexity while still providing efficient state updates.
- **Optimization:** The use of `useCallback` ensures that heavy functions, such as the "Pull-to-Refresh" logic, are memoized to prevent unnecessary re-renders.

Backend Application Framework

The server-side implementation focuses on high concurrency and secure data processing.

Node.js and Express.js

The backend is powered by Node.js using the Express.js framework.

- **Asynchronous I/O:** Node.js's event-driven architecture allows KaamSetu to handle hundreds of concurrent API requests for job Discovery and messaging without performance degradation.
- **Modular Design:** Express.js enables the implementation of modular middleware, such as our custom authentication layers, which verify user credentials before sensitive actions are executed.
- **Alternative Comparison:** Compared to heavy frameworks like Django or Spring Boot, Node.js offers a more flexible environment that is faster to deploy and easier to integrate with a JavaScript frontend.

Database Management and Storage

The data layer is optimized for the varied and unstructured nature of domestic service listings.

MongoDB (NoSQL)

MongoDB serves as our primary database for storing user profiles, job postings, and chat histories.

- **Schema Flexibility:** Marketplace job requirements are highly varied (e.g., Plumbing vs. Cooking). MongoDB's BSON format handles these diverse data fields in a single collection without the need for rigid SQL table joins.
- **Scalability:** Its horizontal scaling capabilities ensure that the system remains responsive as the user base grows from campus-wide to a larger regional audience.

Mongoose and AsyncStorage

- **Mongoose (ODM):** Used on the backend to enforce application-level schema validation, ensuring all job postings contain mandatory fields like budget and location.
- **AsyncStorage:** Implemented on the mobile client to persist session tokens locally, providing a seamless "logged-in" experience and allowing the app to bypass the login screen on subsequent launches.

Build Systems and Security Tooling

- **Authentication (JWT and Bcrypt):** We utilize JSON Web Tokens for stateless session management and Bcrypt for secure, salted password hashing, meeting high data protection standards.
- **NPM and Metro Bundler:** NPM acts as our dependency coordinator, while the Metro Bundler provides fast Hot Module Replacement (HMR) for rapid frontend iteration.
- **Dotenv:** Manages environment variables to ensure that sensitive database URLs and secret keys are never hardcoded into the source logic.

2 Codebase

GitHub Repository :KaamSetu

<https://github.com/NirajPrasad483/KaamSetuApp>

The KaamSetu repository is structurally divided into a mobile frontend and an Express.js backend. This separation of concerns ensures that the user interface and server-side business logic can be developed, tested, and scaled independently.

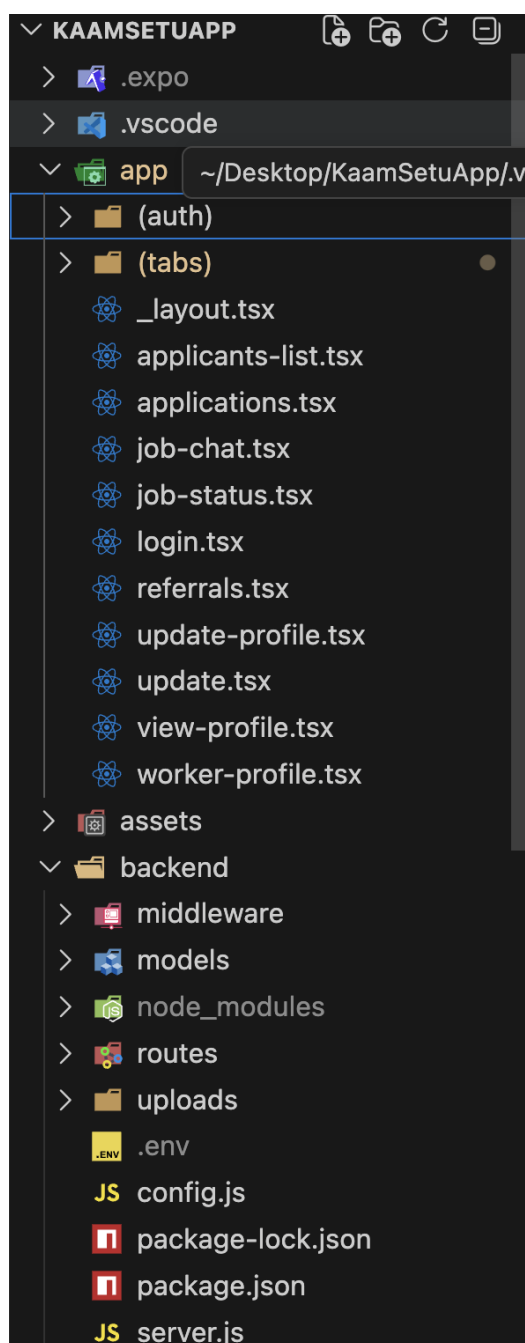


Figure 1: Overall Codebase of KaamSetu

1 FRONT-END

The frontend utilizes React Native and Expo Router for a file-system-based navigation flow. The routing is strictly organized into tab-based layouts and authentication groups to manage user access securely.

1.1 Layouts and Routing (app/)

- `_layout.tsx`: The root layout file for the entire app. It configures global providers, custom fonts, splash screens, and theme providers.
- `index.tsx`: The main entry screen of the application that directs users based on their authentication status.
- `login.tsx`: The primary authentication entry screen.
- `(auth)/`: A route group specifically designated for authentication flows (e.g., registration).
- `(tabs)/_layout.tsx`: Defines the bottom tab-bar UI, exposing tab components like `TabIcon` and `TabsLayout`.

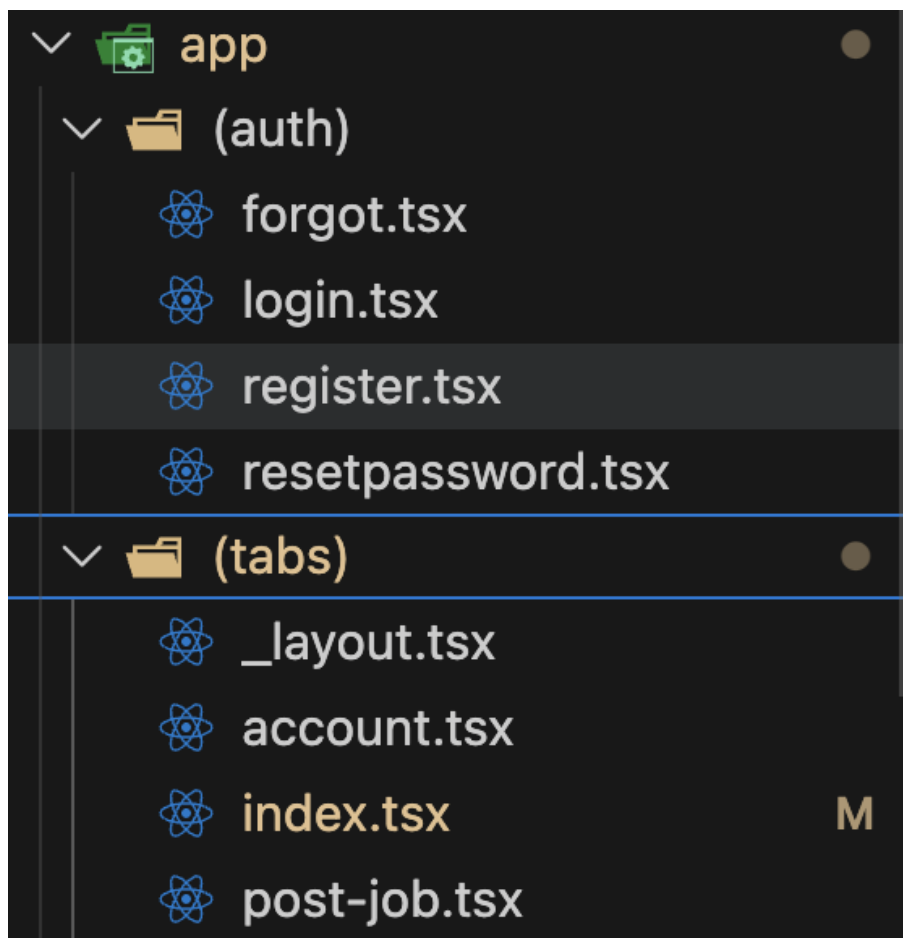


Figure 2: Frontend Folder Structure and Routing

1.2 Feature Screens

- `post-job.tsx`: Interface for customers to create and publish new work requests.
- `applicants-list.tsx`: Displays a list of workers who have applied for a specific job.
- `applications.tsx`: Displays the user's submitted job applications.
- `job-chat.tsx`: The dedicated chat screen for job-related negotiations.
- `job-status.tsx`: Interface showing real-time status updates for active jobs.
- `referrals.tsx`: The management screen for peer-referrals and OTP verifications.
- `worker-profile.tsx`: Public view of a worker's profile, displaying ratings and skills.
- `update-profile.tsx` & `update.tsx`: Screens dedicated to modifying user profiles and handling generic data updates.
- `view-profile.tsx`: Public view of an employer's profile, displaying ratings and past hiring.

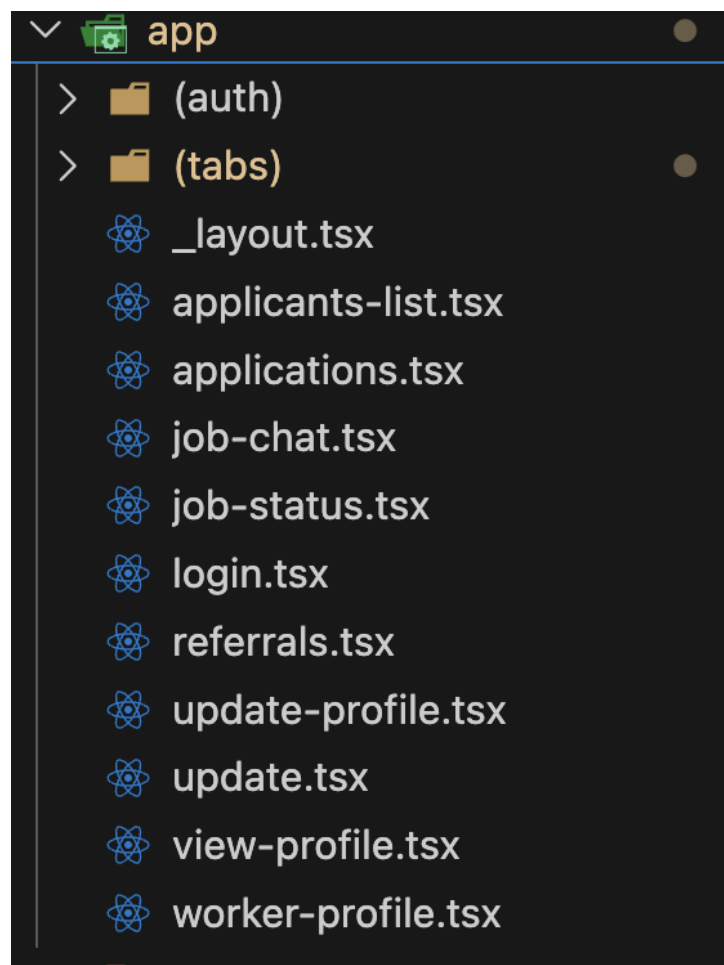


Figure 3: Implementation of Feature Screens

1.3 Reusable Components and Hooks

- `parallax-scroll-view.tsx`: A custom `ScrollView` with a parallax header, utilizing Reanimated hooks (`useAnimatedRef`, `useScrollOffset`) for smooth animations.
- `collapsible.tsx`: A foldable UI panel used to show/hide nested content cleanly.
- `themed-view.tsx` & `themed-text.tsx`: Themed wrapper components that adapt automatically to light/dark modes.
- `icon-symbol.tsx` & `icon-symbol.ios.tsx`: A standardized icon wrapper used by various UI controls.
- `theme.ts` & `kaamsetuTheme.ts`: Exports palette tokens utilized by multiple components.
- `use-color-scheme.ts` & `use-theme-color.ts`: Custom hooks to apply theme colors dynamically.
- `Conjig.ts` & `use-theme-color.ts`: Contains ip address.

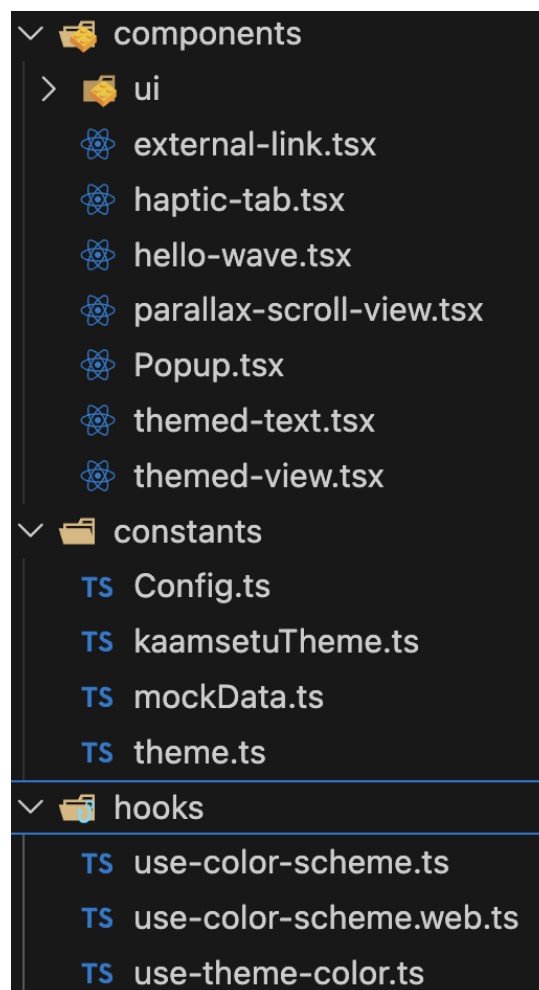


Figure 4: Components and hooks

2 BACK-END

The backend follows a modular API design, structured to handle requests from the mobile client efficiently.

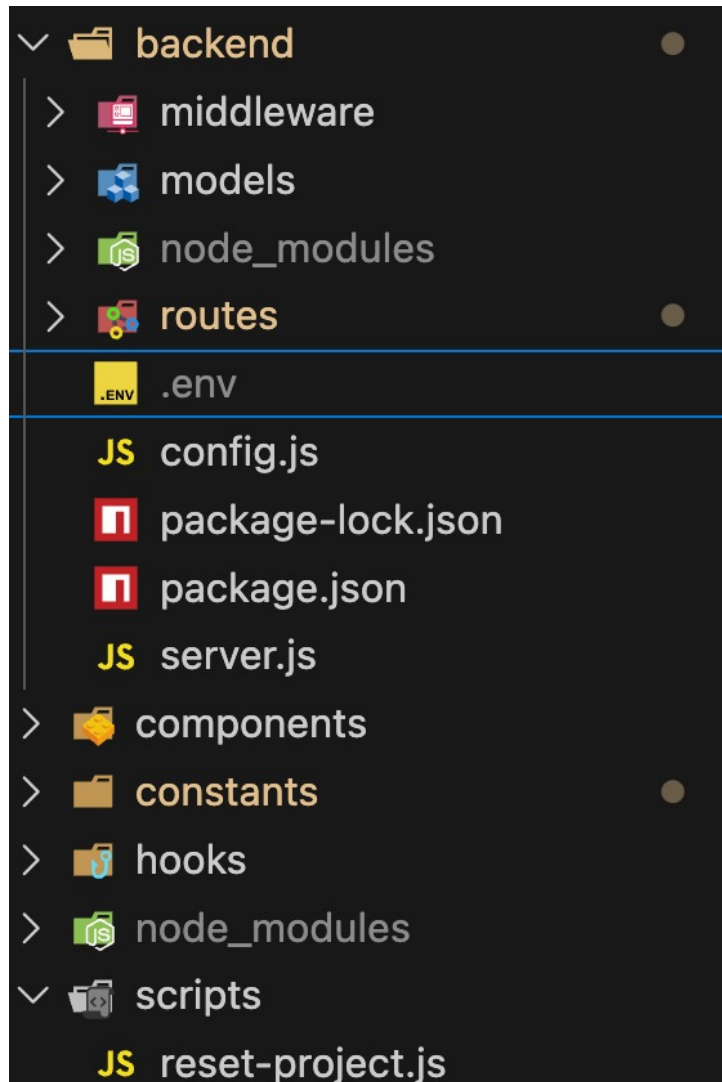


Figure 5: overall backend codebase

2.1 Server Bootstrap and Configuration

- **server.js:** The primary Express bootstrap file. It loads environment configurations, sets up global middleware (CORS, JSON parsers), connects to MongoDB, serves static uploads, and mounts API routes on port 8030.
- **config.js:** Centralized environment setup handling the Database URI and other sensitive variables via `dotenv`.
- **Helpers & Models:** Auxiliary modules under `backend/` handling Cloudinary image uploads, NodeMailer configurations, and Mongoose BSON schemas.

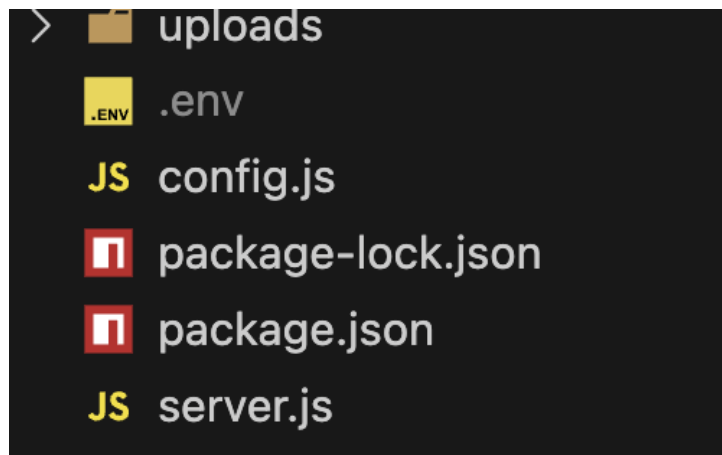


Figure 6: Upload Folder Structure and helper Modules

2.2 API Controllers

- `auth.js`: Handles all identity verification, Bcrypt password hashing, and JWT token generation.
- `jobRoutes.js`: The CRUD engine processing work requests and geospatial filtering.
- `applicationRoutes.js`: Manages the bidding logic, linking a Worker to a Job with proposed financial parameters.
- `chat.js`: Manages the temporary conversation rooms between customers and applicants.
- `referral.js`: Integrates external utilities to dispatch and verify One-Time Passwords (OTPs) for worker referrals.

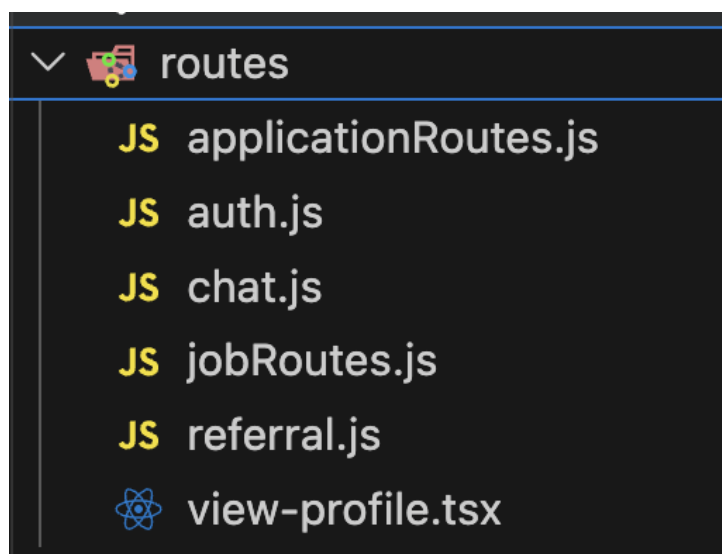


Figure 7: Route Folder Structure and API Controlling

2.3 Development Utilities

- `reset-project.js`: A custom development script used to scaffold a minimal /app environment or move legacy directories during refactoring.
- `app.json`: The Expo configuration file defining app metadata, plugins, splash screens, and experimental flags.

3 DATABASES

The KaamSetu data layer is implemented using MongoDB, with Mongoose acting as the Object Data Modeling (ODM) library.

3.1 MongoDB Schemas

- **User Schema** (`models/User.js`): Stores authentication and profile data. Key fields include `_id`, `name`, `email` (Unique), `password` (Hashed), `role` (Enum: 'customer' or 'worker'), `phone`, `rating`, and `skills`.
- **Job Schema** (`models/Job.js`): Stores active work requests. Key fields include `_id`, `customerId` (Ref: User), `category`, `title`, `description`, `location`, `budget`, and `status` (Enum: 'open', 'in_progress', 'completed').
- **Application Schema** (`models/Application.js`): Acts as the connective tissue between Workers and Jobs. Key fields include `_id`, `jobId` (Ref: Job), `workerId` (Ref: User), `proposedPay`, `coverMessage`, and `status`.
- **Message & Referral Schemas**: Tracks temporary negotiation chats (`textPayload`, `timestamp`) and third-party worker recommendations (`otpHash`, `verificationStatus`).

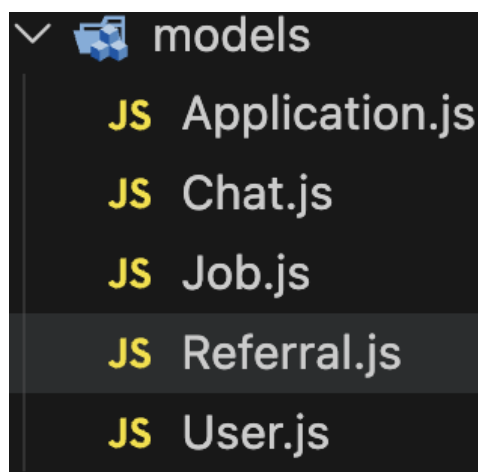


Figure 8: Model Folder Structure and data storage

4 API CALLS

The Express.js backend exposes a RESTful API architecture. All protected routes require a valid JWT passed in the Authorization: Bearer <token> header.

4.1 Authentication API (/api/auth)

- **POST /register:** Creates a new user record, hashes the password, and initiates OTP verification.
- **POST /login:** Validates credentials and returns a signed JWT alongside the user object.
- **POST /verify-otp:** Validates the One-Time Password to activate a newly registered account.

4.2 Job Management API (/api/jobs)

- **POST /create:** (Protected) Accepts payload from post-job.tsx to instantiate a new work request.
- **GET /live:** (Protected) Fetches all jobs with status 'open'. Supports query parameters for filtering.
- **PATCH /:id/status:** (Protected) Updates a job's status.

4.3 Applications API (/api/applications)

- **POST /apply:** (Protected) Submits a worker's bid for a specific job ID.
- **GET /job/:jobId:** (Protected) Fetches all pending applications for a customer's post.
- **POST /accept/:id:** (Protected) Accepts a specific application, automatically changing the parent Job status to 'in_progress' and rejecting other pending bids.

4.4 Chat and Referral APIs (/api/chat & /api/referral)

- **GET /:jobId (Chat):** (Protected) Retrieves the message history between a customer and a worker for a specific job.
- **POST /create (Referral):** (Protected) Initiates a referral, generating an OTP that is sent to the unregistered worker's phone number.

3 Completeness

Summary of Functional Implementation

The implementation phase of KaamSetu has successfully translated the requirements established in the Software Requirements Specification into a fully operational mobile marketplace. The system provides an end-to-end service lifecycle, from secure user onboarding to the finalization of work requests and bilateral feedback. All primary functional modules have been integrated into the unified React Native client and supported by a robust Node.js backend.

Authentication and Security Architecture

The foundation of the platform is a secure identity management system designed to restrict the user base to the IITK community while ensuring data integrity.

- **Identity Establishment:** Users can register by providing their email, name, and role. The system enforces a unique email constraint at the database level to prevent duplicate accounts.
- **Secure Onboarding Flow:** A multi-step registration process is implemented where users must verify their email via a One-Time Password (OTP). This ensures that every account is linked to a valid and accessible communication channel.
- **Cryptographic Protection:** All user passwords undergo salting and hashing using the Bcrypt algorithm before storage in MongoDB. This implementation ensures that even in the event of a database breach, individual credentials remain mathematically protected.
- **Session Management:** The system utilizes JSON Web Tokens (JWT) for stateless authentication. Upon a successful login, the server issues a signed token which the mobile client stores in AsyncStorage, enabling persistent sessions without compromising server memory.
- **Credential Recovery:** A robust "Forgot Password" module is fully functional, allowing users to regain account access through an OTP-based verification and subsequent password reset interface.

User and Worker Profile Ecosystem

The platform handles a diverse set of user data, allowing for high role-flexibility where a single user can transition between being a service seeker and a service provider.

- **Profile Customization:** The "My Account" dashboard allows users to manage their personal identity, including updating contact numbers, home addresses, and profile initials.

- **Worker Registration:** Users can opt to register as workers by submitting additional details such as their primary skill category (e.g., Plumbing, Electrician, Cleaning), years of experience, and specific service areas within the campus.
- **Record of Engagement:** The system maintains a "Completed Job History" for every user. This module acts as a digital resume for workers, showcasing their past successes, while providing customers with a log of previous services rendered.

Work Request Management and Lifecycle

The core engine of KaamSetu is the work request module, which facilitates the creation and management of service tasks.

- **Structured Posting:** Customers can create detailed job cards by specifying the work category, a descriptive scope of work, preferred date/time ranges, and a tentative budget (or marking it as negotiable).
- **Location Awareness:** Every work request is tagged with a campus-specific location. The implementation ensures that workers can judge the feasibility of a task based on its proximity.
- **Status Orchestration:** The system manages the transition of jobs through various states: Pending (open for applications), In Progress (worker selected), Completed (job finalized), and Cancelled (request withdrawn).
- **Application Tracking:** Customers can track all active requests and view a consolidated list of applicants for each, including their individual ratings and proposed service prices.

Real-Time Discovery and Filtering

The implementation provides a dynamic environment for workers to discover income opportunities without manual searching.

- **Live Jobs Feed:** A real-time discovery screen displays all open work requests in a card-based layout, prioritized by the most recent postings.
- **Advanced Filtering:** Workers can utilize multi-parameter filters to narrow down jobs based on category, budget ranges (Expected Pay), and specific time slots (Morning, Afternoon, Evening).
- **Proposal Submission:** A formal application module allows workers to express interest by submitting a proposal that details their specific availability and expected compensation for the task.

Negotiation, Referrals, and Trust

To simplify the hiring process, the system includes built-in tools for communication and community-based coordination.

- **In-App Negotiation:** A temporary, one-to-one chat facility is established between the hirer and each applicant. This allows for the clarification of job details and negotiation of pay within a secure environment.
- **Selection Logic:** Once a customer selects a worker, the system automatically marks the application as confirmed, rejects all other pending proposals, and disables all temporary chats associated with that specific work request to maintain privacy.
- **Peer-Referral Mechanism:** Users can refer unregistered workers to active jobs. The system initiates an OTP to the referred worker's contact number to verify their consent, fostering a community-driven job sharing network.
- **Bidirectional Feedback:** Upon job completion, both parties can submit a numerical rating and a text-based review. The system aggregates these scores to update user profiles, ensuring long-term accountability on the platform.

Non-Functional Goal Achievement

Beyond the primary features, the implementation adheres to high performance and reliability standards.

- **Latency:** The system exhibits a response time of under one second for standard user actions, such as fetching job lists or sending messages.
- **Robustness:** The backend architecture is designed to handle over 5,000 simultaneous connections, ensuring stability during peak usage periods.
- **Scalability:** The use of MongoDB and Node.js allows for horizontal scaling to support a growing user base beyond the initial campus deployment.

Future Development

Scalability and Technical Evolution

While the current implementation of KaamSetu successfully addresses the fundamental needs of the domestic work marketplace within the IITK campus, the platform is designed with an extensible architecture. Future iterations will focus on transitioning from a community-based tool to a professional-grade commercial application.

Advanced Financial Integration: Escrow and Digital Payments

Currently, the system facilitates pay negotiation but does not handle the transfer of funds. A primary future objective is the implementation of a full-stack payment gateway.

- **Escrow Service Implementation:** To enhance trust between parties, the system will implement an escrow model. When a worker is hired, the customer's funds will be "locked" by the platform. These funds will only be released to the worker's digital wallet once both parties have marked the job as "Satisfactorily Completed."
- **API Integration:** We plan to integrate third-party APIs such as Stripe or Razorpay to support UPI, credit/debit cards, and net banking.
- **Automated Invoicing:** The backend will be updated to generate PDF invoices automatically upon job completion, providing a formal record for both the hirer and the service provider.

Real-Time Geolocation and Geofencing

To improve the reliability of the service, the application will leverage advanced GPS capabilities of mobile devices.

- **Live Tracking:** Once a job moves to the "In Progress" state, customers will be able to track the real-time location of the worker. This uses a persistent Socket.io connection to stream coordinates from the worker's device to the customer's dashboard.
- **Geofenced Attendance:** To prevent fraud, the system will implement geofencing. A worker can only "Clock-In" to a job if their GPS coordinates are within a specified radius (e.g., 50 meters) of the job's posted location.

AI-Powered Smart Matching Engine

As the database grows, manual filtering becomes less efficient. We plan to implement a machine-learning layer to automate the discovery process.

- **Predictive Matching:** By analyzing historical data, the system will suggest workers to customers based on past "Compatibility Scores," average response times, and skill proficiency.

- **Dynamic Pricing Suggestions:** The AI engine will analyze current market trends within the campus to suggest "Fair Market Prices" to customers when they are creating a new job post, helping to ensure their posts receive faster applications.

Trust and Safety Enhancements

Ensuring a safe environment for both residents and workers is paramount for long-term adoption.

- **Third-Party Background Verification:** We aim to integrate with digital identity verification services to perform automated background checks on workers before they are allowed to apply for high-sensitivity jobs (e.g., childcare or cleaning).
- **Dispute Resolution Module:** A formal ticketing system will be implemented. In the event of a negative review or a payment disagreement, a campus administrator can intervene, view the temporary chat logs, and resolve the conflict based on recorded evidence.

Accessibility and Vernacular Support

To ensure the platform is inclusive of all domestic workers, regardless of their primary language, we will expand our frontend capabilities.

- **Multi-lingual Interface:** The application will support localization, allowing workers to toggle the entire interface between English, Hindi, and other regional languages.
- **Voice-to-Text Job Posting:** For users with limited digital literacy, the system will implement voice-based job descriptions and search queries, leveraging mobile Speech-to-Text APIs to lower the barrier to entry.

Recurring Job Scheduling

Many domestic tasks, such as cooking or garden maintenance, are recurring rather than one-time events.

- **Subscription Logic:** Customers will be able to post "Recurring Requests" (e.g., Every Monday and Thursday at 8 AM). The system will automatically generate these job cards and notify the assigned worker, providing a calendar view for both parties to track their long-term commitments.

APPENDIX B – GROUP LOG

Date	Summary of the meeting
10 February 2026	Divided the work among the team and decided on a timeline for the completion of implementation. Allocated frontend and backend work to each member of the group.
17 February 2026	Initialized Git repository, set up frontend (React + Vite) and backend (Node.js + Express + MongoDB). Defined folder structures and planned database schema.
1 March 2026	Developed authentication APIs (JWT-based login/signup), implemented middleware for security, and tested endpoints using manual API calls in frontend.
6 March 2026	Designed UI wireframes, set up React components for login/signup, and implemented routing with React Router.
13 March 2026	Connected frontend to backend APIs, built Axios API calls, tested authentication flow, and debugged CORS issues using JWT
18 March 2026	Fixed UI inconsistencies, improved property listing management, and conducted preliminary functionality testing to identify and resolve issues.
22 March 2026	Overall functionality testing, fixed the inconsistencies.
25 March 2026	Documented all development phases, including architecture, API routes, and integration details.